

Exame AI-901

Guia de Estudo Intensivo

Atualizado para as "Skills Measured" de 15 de abril de 2026

O sucessor do AI-900 — foco total no Microsoft Foundry

Feito para: Lucas — prova na quinta-feira de manhã

Formato: seções curtas de 5–10 min, para ler entre tarefas

Sem prática no Azure? As seções do Foundry descrevem as telas e o código que você precisa **reconhecer** — não decorar.

Domínio 1 · Conceitos de IA · 40–45%

Domínio 2 · Microsoft Foundry · 55–60%

Nota de corte: 700

Conteúdo do guia: 1. Raio-X da prova · 2. Domínio 1 · 3. Domínio 2 · 4. Simulado comentado · 5. Cheat sheet de véspera

1 · Raio-X da prova

Estrutura

Item	Detalhe
Domínio 1	Identificar conceitos e capacidades de IA — 40–45%
Domínio 2	Implementar soluções de IA usando o Microsoft Foundry — 55–60%
Nota de corte	700 (escala 1–1000)
Tipos de questão	Múltipla escolha, seleção múltipla, arrastar-e-soltar, completar frase, "hot area". Sem laboratório ao vivo — por isso este guia em texto funciona.
Perfil esperado	Início de carreira em desenvolvimento de soluções de IA. Espera-se noção de sintaxe Python e familiaridade com recursos do Azure, REST APIs e SDKs.

NÃO ESTUDE MAIS ISSO (SAIU DO EXAME)

Detecção de anomalias · Regressão / classificação / clustering · Azure ML Designer e AutoML · LUIS (Language Understanding) · Azure Bot Service · QnA Maker · Knowledge mining / Cognitive Search como workload. Se seu material fala disso, **pule**.

Mapa de renomeações (cai em prova!)

Nome antigo (material desatualizado)	Nome atual (o que a prova usa)
Azure AI Studio → Azure AI Foundry	Microsoft Foundry (rebatizado no Ignite 2025; oficial desde jan/2026)
Azure Cognitive Services → Azure AI Services	Foundry Tools (Speech, Language, Vision, Content Understanding...)
Form Recognizer → Document Intelligence	Motor absorvido pelo Azure Content Understanding
Portal antigo baseado em hubs	Foundry (classic)

PEGADINHA

Se uma alternativa citar "Azure AI Studio" ou "Cognitive Services" como resposta para algo *atual*, desconfie: a prova prefere a nomenclatura nova. O Foundry hoje hospeda modelos da OpenAI, **Anthropic (Claude)**, Meta, Mistral, DeepSeek, xAI e Microsoft (Phi) — não é só OpenAI.

Como usar este guia até quinta

- **Terça:** seções 2 (Domínio 1) e 3.1–3.5 (Foundry: portal, prompts, SDK, agentes).
- **Quarta:** seções 3.6–3.9 (fala, visão, Content Understanding) + simulado às ~15h + cheat sheet à noite. Nada novo depois das 20h.
- **Quinta, no café:** só o cheat sheet (últimas 2 páginas). Não faça simulado no dia.

2.1 · IA Responsável — os 6 princípios

Base de várias questões fáceis. O truque é distinguir princípios parecidos pelo **cenário**:

Princípio	Ideia central	Cenário típico de prova
Justiça (Fairness)	O sistema não pode favorecer/prejudicar grupos (gênero, etnia, idade...)	Modelo de aprovação de crédito rejeita mais um grupo demográfico → problema de justiça
Confiabilidade e segurança (Reliability & safety)	Funcionar de forma previsível, mesmo em condições inesperadas; testes rigorosos	Carro autônomo, diagnóstico médico → precisa de testes extensivos antes de liberar
Privacidade e segurança (Privacy & security)	Proteger dados pessoais usados no treino e no uso	Sistema treinado com dados de pacientes deve anonimizar/proteger os dados
Inclusão (Inclusiveness)	IA deve empoderar todos , inclusive pessoas com deficiência	App que descreve imagens para pessoas cegas; legendas automáticas para surdos
Transparência (Transparency)	Usuários devem entender como o sistema funciona e suas limitações	Informar que a resposta veio de uma IA e explicar por que a decisão foi tomada
Responsabilização (Accountability)	Pessoas respondem pelo sistema; governança e supervisão humana	Empresa define comitê/processo de revisão humana para decisões da IA

PEGADINHA CLÁSSICA

Transparência = explicar o funcionamento ao usuário. **Responsabilização** = humanos/empresa assumem as consequências e supervisionam. **Justiça** = tratar grupos igualmente. **Inclusão** = alcançar quem tem necessidades diferentes (acessibilidade). Não confunda os pares!

2.2 · Como funcionam os modelos generativos

- **LLM (Large Language Model)**: rede neural (arquitetura **transformer**) treinada em enormes volumes de texto. Processa **tokens** (pedaços de palavra) e prevê o próximo token repetidamente — é assim que "escreve".
- **Embeddings**: vetores numéricos que representam o *significado* de um texto. Usados em busca semântica e RAG.
- **Grounding / RAG (Retrieval-Augmented Generation)**: anexar dados externos (documentos da empresa) ao prompt para o modelo responder com base neles — reduz alucinação sem retreinar o modelo.
- **Fine-tuning**: retreinar parcialmente o modelo com dados próprios. Mais caro que RAG; use quando precisa mudar o *comportamento*, não só o *conhecimento*.
- **Multimodal**: modelo que aceita/gera mais de uma modalidade (texto + imagem + áudio), ex.: GPT-4o.

Parâmetros de configuração (decore a função de cada um)

Parâmetro	O que controla	Regra prática
temperature	Aleatoriedade/criatividade da resposta	Baixa ($\approx 0-0,3$) = factual e consistente · Alta ($\approx 0,8+$) = criativa e variada
top_p	Também controla aleatoriedade (amostragem por probabilidade acumulada)	Ajuste temperature ou top_p, não os dois
max_tokens	Tamanho máximo da resposta	Resposta cortada no meio? Aumente max_tokens
frequency / presence penalty	Penalizam repetição de tokens/assuntos	Modelo repetitivo? Aumente as penalties

2.3 · Escolha de modelo e opções de deploy

Necessidade do cenário	Escolha
Tarefa complexa, raciocínio, alta qualidade	LLM (ex.: GPT-4o, Claude)
Custo baixo, dispositivo local/edge, tarefa simples	SLM — Small Language Model (ex.: família Phi da Microsoft)
Entender/descrever imagens ou áudio no prompt	Modelo multimodal (GPT-4o)
Gerar imagens novas	Modelo de geração de imagem (DALL-E / GPT-image)
Busca semântica / RAG	Modelo de embeddings

	Serverless / Standard (pay-per-token)	Managed compute
Infra	Nenhuma — Microsoft hospeda	VM/GPU dedicada no seu recurso
Cobrança	Por token consumido	Por hora de computação
Quando usar	Começar rápido, carga variável (padrão da prova)	Modelos open-source customizados, controle total, alto volume constante

2.4 · Workloads de IA — a tabela "cenário → workload"

A prova descreve um cenário e pergunta qual workload é. Este mapa resolve a maioria:

Cenário descrito na questão	Workload correto
Criar conteúdo novo: textos, e-mails, código, imagens	IA generativa
Sistema que executa tarefas de múltiplas etapas sozinho , usando ferramentas (consultar agenda, enviar e-mail, chamar API)	IA agêntica
Classificar sentimento de avaliações; achar nomes/lugares num texto; resumir documentos	Análise de texto
Transcrever reunião; ler texto em voz alta; traduzir fala	Fala (speech)
Classificar fotos, detectar objetos, ler texto em imagem (OCR)	Visão computacional
Extrair campos estruturados de notas fiscais, formulários, gravações, vídeos	Extração de informação

COMO DIFERENCIAR GENERATIVA DE AGÊNTICA

Generativa **produz conteúdo** quando pedida. Agêntica **age**: decide etapas, usa ferramentas e conclui um objetivo com autonomia. "Chatbot que redige e-mail" = generativa. "Assistente que lê a agenda, encontra horário e envia o convite sozinho" = agêntica.

2.5 · Técnicas de análise de texto

Técnica	O que faz	Exemplo
Extração de palavras-chave (key phrases)	Identifica os termos principais do texto	"entrega atrasada", "produto danificado"
Deteção de entidades (NER)	Localiza e classifica entidades: pessoas, lugares, datas, organizações	"Lucas" = Pessoa; "Limeira" = Local; "12/08" = Data
Análise de sentimento	Classifica como positivo / negativo / neutro / misto, com score	Monitorar satisfação em avaliações de clientes
Sumarização	Gera resumo (extrativo: frases originais; abstrativo: texto novo)	Resumir um contrato de 40 páginas

PEGADINHA

"Descobrir **do que** o texto fala" = palavras-chave/entidades. "Descobrir **como o autor se sente**" = sentimento.
"Encurtar mantendo o essencial" = sumarização.

2.6 · Fala, visão e extração multimodal

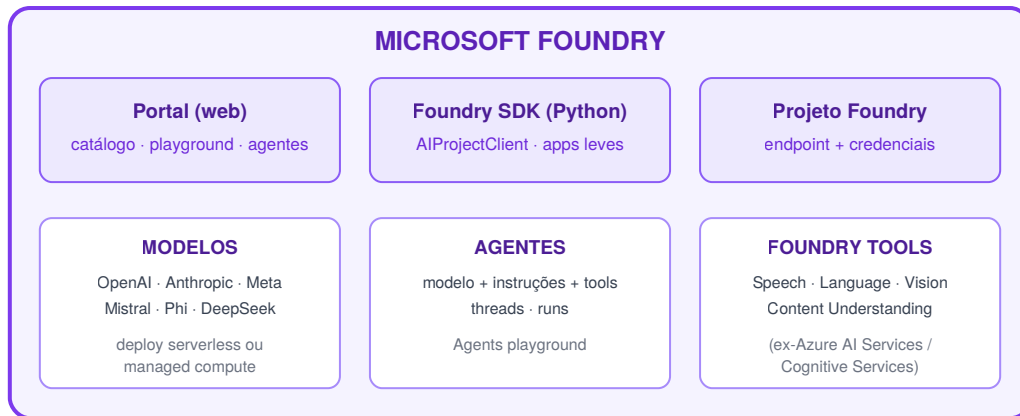
- **Reconhecimento de fala** (speech-to-text): áudio → texto. Usos: transcrição, legendas ao vivo, ditado.
- **Síntese de fala** (text-to-speech): texto → áudio com vozes neurais; **SSML** ajusta tom, velocidade e pronúncia.
- **Tradução de fala**: reconhece, traduz e pode sintetizar no idioma de destino, quase em tempo real.
- **Visão computacional**: classificação de imagem (rótulo para a imagem inteira), detecção de objetos (o quê + **onde**, com caixas), OCR (texto em imagens). Modelos multimodais hoje fazem descrição e perguntas sobre imagens direto no prompt.
- **Geração de imagem**: criar imagens novas a partir de descrição em texto (DALL-E / GPT-image).
- **Extração de informação**: transformar conteúdo **não estruturado** (documentos, imagens, áudio, vídeo) em dados **estruturados** — na prática, é o Azure Content Understanding (seção 3.9).

PEGADINHA

Classificação de imagem responde "isto É um gato?". Detecção de objetos responde "ONDE estão os gatos na foto?" (bounding boxes). OCR responde "que TEXTO há na foto?".

3.1 · O que é o Microsoft Foundry

É a plataforma unificada da Microsoft para **construir, implantar e gerenciar apps e agentes de IA** — a "fábrica de apps e agentes". Reúne num único lugar: catálogo de modelos, deployments, playgrounds, agentes, avaliação/observabilidade e os **Foundry Tools** (os antigos serviços de IA: Speech, Language, Vision, Content Understanding). Acessível pelo **portal web** e pelo **Foundry SDK** (Python).



Arquitetura mental do Foundry: duas portas de entrada (portal e SDK), três blocos de capacidade.

3.2 · Tour virtual: implantar um modelo e usar o playground

Você não vai praticar, então grave o **fluxo** — a prova pergunta a ordem e o vocabulário das etapas:

1. **Criar um projeto** no portal do Foundry (o projeto agrupa modelos, agentes e conexões, e fornece o **endpoint**).
2. Abrir o **catálogo de modelos** (Model catalog), filtrar/comparar e escolher o modelo.
3. Clicar em **Deploy** → dar um **nome de deployment** → escolher o tipo (serverless/standard é o padrão). O app chamará o modelo pelo **nome do deployment**, não pelo nome do modelo.
4. Abrir o **Chat playground**: à esquerda, a caixa de **system message** (instruções/persona); ao centro, o chat; à direita, os **parâmetros** (temperature, top_p, max_tokens).
5. Botão **"View code"**: gera o código de exemplo com endpoint, chave e deployment já preenchidos — é a ponte do playground para o app.

PEGADINHA

"Onde você testa um modelo interativamente antes de escrever código?" → **playground do portal**. "O que o código usa para referenciar o modelo?" → o **nome do deployment**. "Modelo respondeu cortado no playground?" → aumente **max_tokens**.

3.3 · Prompts eficazes (system vs. user)

Tipo	Papel	Exemplo
System prompt	Define comportamento, persona, regras e formato — vale para a conversa toda	"Você é um assistente de suporte. Responda em português, em até 3 frases, e nunca invente preços."
User prompt	A pergunta/tarefa do usuário naquele turno	"Qual o prazo de entrega para Limeira?"

Boas práticas que a prova cobra: ser específico e claro; dar contexto/dados (grounding); definir formato de saída ("responda em JSON"); **few-shot** = incluir exemplos de entrada→saída no prompt (zero-shot = sem exemplos); quebrar tarefas complexas em etapas; usar o system prompt para restrições, não o user prompt.

3.4 · Foundry SDK — o código que você precisa RECONHECER

A prova de fundamentos não pede código de cabeça: ela mostra trechos e pergunta o que fazem ou qual linha completa. Grave a **anatomia**: `endpoint` + `credential` → cliente do projeto → chamada de chat com `model` (deployment) e lista de `messages` com `role`.

```
# pip install azure-ai-projects azure-identity
from azure.ai.projects import AIProjectClient
from azure.identity import DefaultAzureCredential

project = AIProjectClient(
    endpoint="https://meu-recurso.services.ai.azure.com/api/projects/meu-projeto",
    credential=DefaultAzureCredential() # autenticação Entra ID, sem chave no código
)

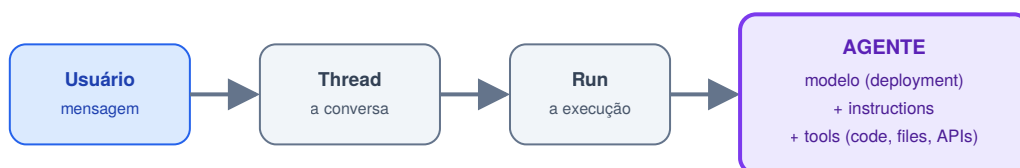
chat = project.get_openai_client() # cliente de chat do projeto
resposta = chat.chat.completions.create(
    model="gpt-4o", # NOME DO DEPLOYMENT
    messages=[
        {"role": "system", "content": "Você é um assistente conciso."},
        {"role": "user", "content": "Explique RAG em 2 frases."}
    ],
    temperature=0.3,
    max_tokens=300
)
print(resposta.choices[0].message.content)
```

DECORE AS IMPRESSÕES DIGITAIS

`DefaultAzureCredential` = autenticação sem expor chave · `role: system/user/assistant` = papéis das mensagens · `choices[0].message.content` = onde está a resposta · histórico de conversa = **reenviar as mensagens anteriores** na lista (o modelo não tem memória própria).

3.5 · Agentes: portal e código

Agente = modelo + instruções + ferramentas (tools). A conversa vive numa **thread**; cada execução é um **run**. No portal, o **Agents playground** permite criar e testar um agente sem código: você define nome, escolhe o deployment do modelo, escreve as **instructions** (o "system prompt do agente") e adiciona tools — ex.: **code interpreter** (executa Python para cálculos/gráficos), **file search / knowledge** (busca em arquivos que você anexa) e **actions/functions** (chamar APIs externas).



Vocabulário que a prova usa: agente, thread (conversa persistente), run (execução), tools.

```
# criar e usar um agente com o Foundry SDK
agente = project.agents.create_agent(
    model="gpt-4o",
    name="agente-suporte",
    instructions="Você ajuda clientes com dúvidas sobre pedidos."
)
thread = project.agents.threads.create() # nova conversa
project.agents.messages.create(thread_id=thread.id,
    role="user", content="Onde está meu pedido 123?")
run = project.agents.runs.create_and_process( # executa o agente
    thread_id=thread.id, agent_id=agente.id)
```

CENÁRIO DE PROVA

"Um analista sem experiência em código precisa criar e testar um agente" → **Agents playground no portal do Foundry**. "O agente precisa fazer cálculos sobre um CSV enviado" → tool **code interpreter**. "O agente deve responder com base nos PDFs internos da empresa" → tool de **file search/knowledge** (grounding).

Conexão com o que você já faz: é o mesmo padrão de tool calling do seu TCC Sentinel — o LLM decide qual ferramenta chamar e o backend executa. Se entender o Sentinel, você entende agentes no Foundry.

3.6 · Texto e fala com o Foundry

Análise de texto num app leve

Duas rotas possíveis — a prova aceita ambas as ideias: (1) usar um **modelo generativo** com um prompt de análise ("classifique o sentimento e liste as entidades deste texto, em JSON"); (2) usar o **serviço Language nos Foundry Tools**, com operações prontas de sentimento, entidades, key phrases e sumarização.

Azure Speech nos Foundry Tools

```
# padrão clássico do Azure Speech (reconheça as classes!)
import azure.cognitiveservices.speech as speechsdk

config = speechsdk.SpeechConfig(subscription=chave, region=regiao)

# FALA → TEXTO (reconhecimento)
rec = speechsdk.SpeechRecognizer(speech_config=config)
resultado = rec.recognize_once()          # captura uma fala do microfone

# TEXTO → FALA (síntese)
sint = speechsdk.SpeechSynthesizer(speech_config=config)
sint.speak_text_async("Olá, Lucas!").get()
```

DECORE

`SpeechRecognizer` = ouvir (STT) · `SpeechSynthesizer` = falar (TTS) · **SSML** = marcação XML para controlar voz, tom e velocidade na síntese · a voz é escolhida via `speech_synthesis_voice_name`.

Responder a prompts falados com modelo multimodal

Conceito novo do AI-901: em vez de encadear STT → LLM → TTS, um **modelo multimodal implantado** (ex.: GPT-4o com áudio / realtime) recebe o **áudio diretamente** como entrada e pode responder em texto ou áudio. Na prova: "app de voz com a menor latência/menos componentes" → modelo multimodal direto; "só transcrever uma reunião" → Speech (STT) basta.

3.7 · Visão e geração de imagem

Interpretar imagem no prompt (modelo multimodal)

```
resposta = chat.chat.completions.create(
    model="gpt-4o",
    messages=[{
        "role": "user",
        "content": [
            {"type": "text", "text": "O que há de errado nesta peça?"},
            {"type": "image_url", "image_url": {"url": url_da_foto}}
        ]
    }]
)
```

Impressão digital: o `content` vira uma **lista** misturando blocos `text` e `image_url`. Se a questão mostrar isso, é **entrada visual em prompt multimodal**.

Gerar imagens novas

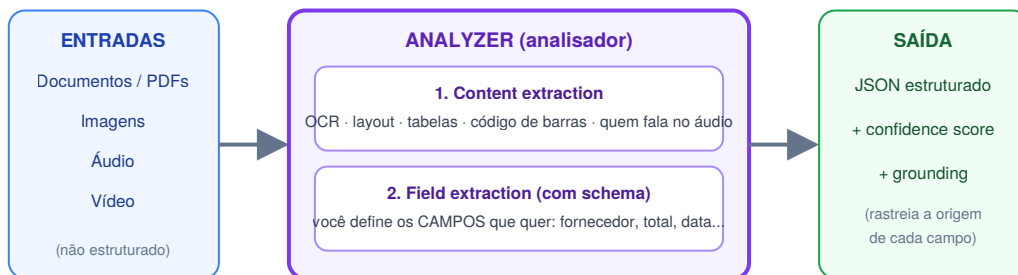
```
img = chat.images.generate(
    model="dall-e-3",          # deployment de geração de imagem
    prompt="logotipo minimalista de uma cafeteria, tons terrosos",
    size="1024x1024"
)
```

PEGADINHA

"ANALISAR/entender uma imagem existente" → modelo **multimodal** (imagem na entrada). "CRIAR uma imagem nova a partir de texto" → modelo de **geração de imagem** (imagem na saída). A questão sempre esconde o verbo.

3.8 · Azure Content Understanding (Foundry Tools) — extração de informação

Serviço que usa IA generativa para transformar conteúdo **não estruturado** — documentos, imagens, áudio e vídeo — em **saída estruturada definida por você**. Sucessor prático do Document Intelligence/Form Recognizer, agora multimodal.



Pipeline do Content Understanding: 4 modalidades entram, um analyzer com schema processa, JSON confiável sai.

Conceito	O que significa (e como cai)
Analyzer	A unidade de configuração. Há pré-construídos (nota fiscal/invoice, recibo, documentos de identidade) e customizados (você define o schema para o SEU layout)
Schema de campos	Lista dos campos a extrair/classificar/gerar. "Preciso do CNPJ, data e total de cada nota" = definir schema com esses 3 campos
Confidence score	Cada campo extraído vem com um grau de confiança → permite mandar para revisão humana só o que estiver abaixo do limiar (cenário clássico de prova)
Grounding	Cada valor aponta para onde no documento/mídia ele foi encontrado — rastreabilidade e auditoria
Áudio/vídeo	Transcrição com identificação de falantes; resumo de chamadas de call center; descrição de cenas e extração de eventos em vídeo

CENÁRIOS QUE APONTAM PARA CONTENT UNDERSTANDING

"Extrair automaticamente campos de milhares de notas fiscais" · "Processar formulários digitalizados manuscritos" · "Analisar gravações do call center e extrair motivo da ligação" · "Extrair informações-chave de vídeos de treinamento" · "Reduzir revisão manual usando scores de confiança". Palavra-chave: **não estruturado → estruturado**.

3.9 · Qual ferramenta para qual tarefa? (resumo do Domínio 2)

A questão pede...	Resposta
Testar modelo sem código, ajustar system prompt e parâmetros	Chat playground do portal Foundry
App de chat leve em Python	Foundry SDK (AIProjectClient + chat completions)
Assistente que usa ferramentas / executa tarefas	Agente (portal Agents playground ou SDK)
Sentimento, entidades, resumo de textos	Modelo generativo com prompt ou Language (Foundry Tools)
Transcrever / sintetizar / traduzir fala	Azure Speech (Foundry Tools)
App de voz conversando direto com o modelo	Modelo multimodal com entrada de áudio
Entender/descrever imagem existente	Modelo multimodal (imagem no prompt)
Criar imagem nova a partir de texto	Modelo de geração de imagem (DALL-E / GPT-image)
Documentos, formulários, áudio, vídeo → dados estruturados	Azure Content Understanding (Foundry Tools)

4 · Simulado rápido (16 questões)

Faça na quarta ~15h, sem consultar. Gabarito comentado na sequência. Proporção parecida com a prova: 6 questões do Domínio 1, 10 do Domínio 2.

Q1. Um banco descobre que seu modelo de crédito aprova menos empréstimos para mulheres com o mesmo perfil financeiro que homens. Qual princípio de IA Responsável foi violado?

A) Transparência B) Justiça C) Responsabilização D) Confiabilidade e segurança

Q2. Um aplicativo descreve em áudio o conteúdo de fotos para usuários com deficiência visual. Esse app é exemplo principalmente de qual princípio?

A) Inclusão B) Justiça C) Privacidade e segurança D) Transparência

Q3. As respostas de um chatbot estão criativas demais e inconsistentes para um caso de suporte técnico. Qual parâmetro você reduz?

A) max_tokens B) frequency penalty C) temperature D) número de mensagens

Q4. Você precisa rodar um modelo de linguagem em um dispositivo com pouca capacidade, para uma tarefa simples e com custo mínimo. Qual é a melhor escolha?

A) GPT-4o B) Um SLM como o Phi C) Um modelo de embeddings D) DALL-E

Q5. Um assistente recebe a meta "organize minha semana": consulta a agenda, identifica conflitos, remarca reuniões e envia os convites — tudo sozinho. Qual workload descreve isso?

A) IA generativa B) Análise de texto C) IA agêntica D) Extração de informação

Q6. Uma empresa quer saber se as avaliações dos clientes no site são positivas ou negativas. Qual técnica de análise de texto usar?

A) Sumarização B) Detecção de entidades C) Extração de palavras-chave D) Análise de sentimento

Q7. No portal do Microsoft Foundry, qual é a ordem correta para começar a usar um modelo generativo?

A) Playground → catálogo → deploy B) Catálogo de modelos → deploy → playground C) Deploy → catálogo → SDK D) SDK → playground → deploy

Q8. No código do app, qual valor é usado no parâmetro `model` da chamada de chat?

A) O nome do modelo no catálogo B) O ID do projeto C) O nome do deployment D) A região do recurso

Q9. Você quer que o chatbot sempre responda em português formal e nunca fale de concorrentes, em toda a conversa. Onde colocar essas regras?

A) No user prompt de cada mensagem B) No system prompt C) No parâmetro top_p D) No nome do deployment

Q10. Incluir 3 exemplos de pergunta→resposta dentro do prompt para orientar o formato da saída é uma técnica chamada:

A) Fine-tuning B) Grounding C) Few-shot prompting D) Embedding

Q11. No código `AIPProjectClient(endpoint=..., credential=DefaultAzureCredential())`, o que a classe `DefaultAzureCredential` faz?

A) Define o modelo padrão B) Autentica via Microsoft Entra ID sem chave no código C) Criptografa as mensagens D) Define a região do recurso

Q12. Um agente no Foundry precisa fazer cálculos estatísticos sobre um arquivo CSV enviado pelo usuário. Qual tool adicionar ao agente?

A) File search B) Code interpreter C) Web search D) SSML

Q13. No SDK de agentes, o que representa uma **thread**?

A) O modelo implantado B) A conversa persistente com suas mensagens C) Uma execução única do agente D) A ferramenta de busca em arquivos

Q14. Num trecho de código, o campo `content` de uma mensagem é uma lista com um bloco `{"type": "text" ...}` e um bloco `{"type": "image_url" ...}`. O que esse app faz?

A) Gera uma imagem nova B) Envia uma imagem para o modelo multimodal interpretar C) Extrai campos de um formulário D) Converte texto em fala

Q15. Uma transportadora recebe 5.000 notas fiscais por mês em PDF, com layouts variados, e quer extrair automaticamente CNPJ, data e valor total, enviando para revisão humana apenas os casos incertos. Qual solução do Foundry atende melhor?

A) Azure Speech B) Chat playground C) Azure Content Understanding com analyzer customizado e confidence scores D) Modelo de geração de imagem

Q16. Qual das opções descreve corretamente a diferença entre deploy serverless (standard) e managed compute?

A) Serverless cobra por hora; managed por token B) Serverless cobra por token e dispensa infraestrutura; managed usa computação dedicada cobrada por hora C) Managed é sempre mais barato D) Serverless só funciona com modelos open-source

Gabarito comentado

Q1: B. Tratamento desigual entre grupos = Justiça. Transparência seria não explicar a decisão; responsabilização, ausência de supervisão humana.

Q2: A. Tecnologia que empodera pessoas com deficiência = Inclusão. É o exemplo canônico da Microsoft.

Q3: C. Temperature controla aleatoriedade/criatividade. Para suporte técnico, valores baixos (0–0,3) dão consistência. Max_tokens só limita o tamanho.

Q4: B. SLM (Small Language Model, ex.: Phi) = leve, barato, roda em edge. GPT-4o é um LLM multimodal grande; embeddings servem para busca; DALL-E gera imagens.

Q5: C. Autonomia + múltiplas etapas + uso de ferramentas para atingir um objetivo = IA agêntica. Se apenas redigisse um texto quando pedido, seria generativa.

Q6: D. Positivo/negativo/neutro = análise de sentimento. Entidades acham nomes/lugares; palavras-chave acham temas; sumarização encurta.

Q7: B. Fluxo do portal: escolher no catálogo → fazer o deploy (com nome de deployment) → testar no playground. Sem deploy não há o que testar.

Q8: C. O código referencia o **nome do deployment** — que pode ser diferente do nome do modelo no catálogo. Pegadinha recorrente.

Q9: B. Regras permanentes de comportamento/persona/restrrição = system prompt. O user prompt muda a cada turno.

Q10: C. Exemplos no prompt = few-shot. Grounding = anexar dados de referência; fine-tuning = retreinar o modelo; embedding = vetor semântico.

Q11: B. DefaultAzureCredential autentica com a identidade do Entra ID (login do dev, managed identity...), evitando chave hard-coded — também é boa prática de IA Responsável (privacidade e segurança).

Q12: B. Code interpreter executa Python para cálculos e gráficos. File search serve para *buscar conteúdo* em documentos (grounding), não calcular.

Q13: B. Thread = a conversa (histórico de mensagens). Run = uma execução do agente sobre a thread. Trio: agente / thread / run.

Q14: B. Content como lista de blocos text + image_url é a assinatura de **entrada visual em prompt multimodal** (analisar imagem existente, não criar).

Q15: C. Não estruturado → estruturado, layouts variados (analyzer customizado com schema), revisão humana seletiva (confidence scores): tudo aponta para Content Understanding.

Q16: B. Serverless/standard = pay-per-token, sem infra própria (padrão para começar). Managed compute = GPU dedicada, cobrança por hora.

Interpretação: 13+ acertos = você está pronto, revise só os erros. 10–12 = releia as seções dos erros na quarta à noite. <10 = priorize as seções 3.2, 3.4, 3.5 e 3.8 — são os pesos-pesados.

5 · Cheat sheet de véspera (quarta à noite + café de quinta)

Nomes atuais

Microsoft Foundry (ex-Azure AI Foundry, ex-Azure AI Studio) · **Foundry Tools** (ex-Azure AI Services / Cognitive Services) · **Foundry (classic)** = portal antigo · Catálogo tem OpenAI, **Anthropic**, Meta, Mistral, Phi (Microsoft), DeepSeek.

IA Responsável em 6 linhas

Justiça = sem viés entre grupos · **Confiabilidade e segurança** = testar muito, funcionar no inesperado · **Privacidade e segurança** = proteger dados · **Inclusão** = acessível a todos (deficiência) · **Transparência** = usuário entende como funciona e os limites · **Responsabilização** = humanos supervisionam e respondem.

Parâmetros

temperature/top_p = criatividade (baixa = factual) · **max_tokens** = tamanho da resposta · **penalties** = anti-repetição · **system prompt** = persona e regras fixas · **few-shot** = exemplos no prompt · **grounding/RAG** = anexar seus dados · **fine-tuning** = retreinar (caro).

Workloads (cenário → nome)

Cria conteúdo = **generativa** · Age sozinho em etapas com ferramentas = **agêntica** · Sentimento/entidades/resumo = **análise de texto** · Transcrever/sintetizar = **fala** · Classificar/detector/OCR em imagem = **visão** · Docs/áudio/vídeo → dados estruturados = **extração de informação**.

Foundry: fluxos e trio do agente

Portal: **catálogo** → **deploy** (nome do deployment!) → **playground** → **View code**. Código chama o modelo pelo **nome do deployment**. Agente = **modelo + instructions + tools** (code interpreter = calcular · file search = responder com base em docs) · **thread** = conversa · **run** = execução. Modelo não tem memória: reenvie o histórico de mensagens.

Impressões digitais de código

`AIProjectClient + DefaultAzureCredential` = Foundry SDK autenticando sem chave · `chat.completions.create(model, messages=[{role, content}])` = chat · content em lista com `image_url` = **analisar imagem** (multimodal) · `images.generate(prompt)` = **criar imagem** · `SpeechRecognizer` = STT · `SpeechSynthesizer` = TTS · `SSML` = controlar a voz.

Content Understanding em 1 parágrafo

Foundry Tool que transforma **documentos, imagens, áudio e vídeo** em JSON estruturado. **Analyzer** (pré-construído ou customizado) + **schema de campos** definidos por você. Cada campo sai com **confidence score** (revisão humana só no incerto) e **grounding** (onde no arquivo o valor estava). Pipeline: content extraction (OCR, layout, tabelas, falantes) → field extraction.

Deploy

Serverless/standard = por token, sem infra, padrão. **Managed compute** = GPU dedicada, por hora, para controle total/open-source custom.

Fora da prova (não perca tempo)

Anomaly detection · regressão/classificação/clustering · ML Designer/AutoML · LUIS · Bot Service · QnA Maker.

QUINTA DE MANHÃ

Leia só esta página no café. Chegue/entre 30 min antes. Nas questões, sublinhe mentalmente o **verbo do cenário** (criar? agir? extrair? entender imagem? gerar imagem?) — ele aponta o workload e a ferramenta. Na dúvida entre duas alternativas, prefira a que usa a **nomenclatura nova** (Foundry, Foundry Tools, Content Understanding). Boa prova!